CS 3244
Machine Learning

Week 8 Post-Class

Backpropagation, Universal Approximation of NN and Regularizing NNs

8 post

Kan Min-Yen

Ang Yi Zhe

Background Photo credits: Rafiq Mirza, Luan Ahn, Rosmarie Voegtli @ Flickr

Recap and Forecast

Understand how validation and regularization complement to address overfitting:

- **Regularization:** a means of restraining the model.
  - L^p norm forms of regularization.

$$J_{test}(h) = J_{train}(h) + \text{overfit penalty}$$

- **Validation**, as a reality check by peeking (at the bottom line).
  - Cross Validation variants: k-fold or leave one out.

Learning outcomes for this week

- Describe the basic idea of Artificial Neural Networks (ANN or NN):
  - Explain how neural networks combine linear units with a non-linear activation function;
  - Explain the topology of a NN;
- Name potential problems with NNs;
- Describe how a neural network is trained using the backpropagation algorithm;
- Name NN regularization techniques;
- Obtain practical experience constructing NNs using PyTorch.

Back Propagation

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Recap: Gradient Descent

GD minimizes:

$$J_{train}(\Theta) = \frac{1}{m}\sum_{j=1}^{m} j(h(\boldsymbol{x}^{(j)}), y^{(j)})$$

By iterative steps along $-\nabla J_{train}$:

$$\Delta\Theta = -\eta\nabla J_{train}(\Theta)$$

Batch GD: $-\nabla J_{train}$ is based on all examples $(\mathbf{X}, \boldsymbol{y})$

SGD: $-\nabla J_{train}$ is based on one example at a time $(\boldsymbol{x}^{(*)}, y^{(*)})$

$$\mathbb{E}_*\left[-\nabla j(h(\boldsymbol{x}^{(*)}), y^{(*)})\right] = -\nabla J_{train}$$

Applying SGD

All the weights $\Theta = \left\{\Theta_{i,j}^{[l]}\right\}$ determine $h(\boldsymbol{x})$

Error on example $\left(\boldsymbol{x}^{(*)}, y^{(*)}\right)$ is

$$j\left(h\left(\boldsymbol{x}^{(*)}\right), y^{(*)}\right) = j(\Theta)$$

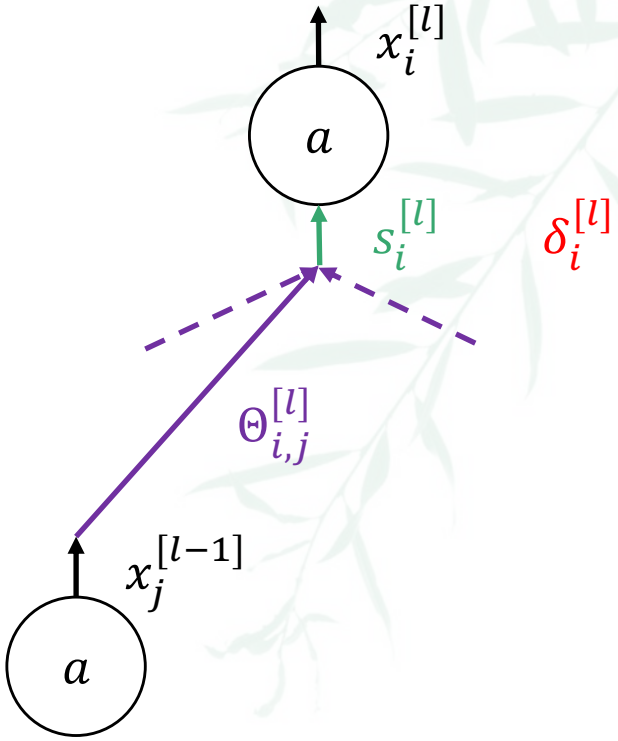
To implement SGD, we need the gradient:

$\nabla j(\Theta)$: $\dfrac{\partial j(\Theta)}{\partial\, \Theta_{i,j}^{[l]}}$ for all i, j, l

Computing $\frac{\partial\ j(\Theta)}{\partial\ \Theta_{i,j}^{[l]}}$

We can evaluate $\frac{\partial\ j(\Theta)}{\partial\ \Theta_{i,j}^{[l]}}$ one by one: analytically or numerically

A trick for efficient computation:

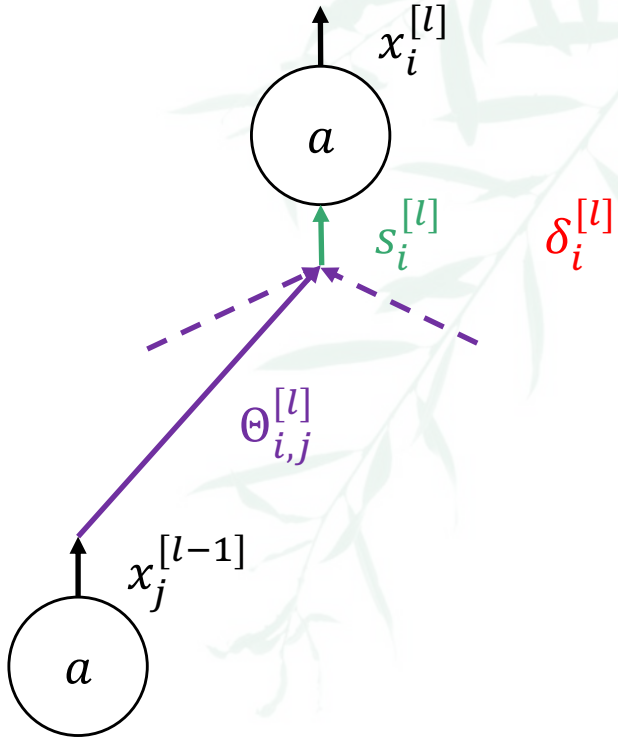


Computing $\frac{\partial\, j(\Theta)}{\partial\, \Theta_{i,j}^{[l]}}$

We can evaluate $\frac{\partial\, j(\Theta)}{\partial\, \Theta_{i,j}^{[l]}}$ one by one: analytically or numerically

A trick for efficient computation:

$$\frac{\partial\, j(\Theta)}{\partial\, \Theta_{i,j}^{[l]}} = \frac{\partial\, j(\Theta)}{\partial\, s_i^{[l]}} \times \frac{\partial\, s_i^{[l]}}{\partial\, \Theta_{i,j}^{[l]}}$$



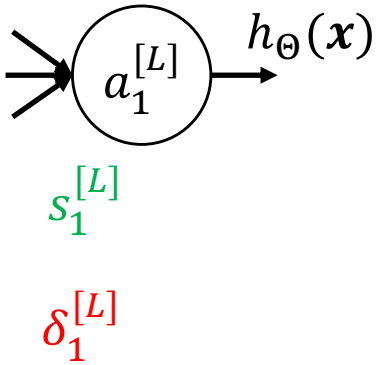
We have $\frac{\partial\, s_i^{[l]}}{\partial\, \Theta_{i,j}^{[l]}} = x_j^{[l-1]}$

We only need: $\frac{\partial\, j(\Theta)}{\partial\, s_i^{[l]}} = \delta_i^{[l]}$

δ for the final layer

$$\delta_i^{[l]} = \frac{\partial j(\Theta)}{\partial s_i^{[l]}}$$

For the final layer $l = L$ and $i = 1$:

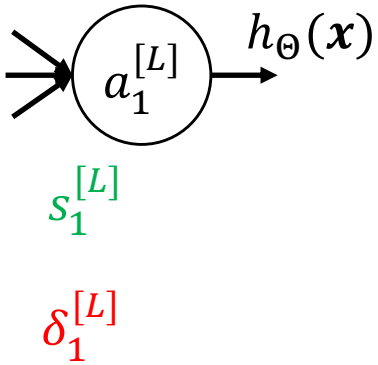


δ for the final layer

$$\delta_i^{[l]} = \frac{\partial j(\Theta)}{\partial s_i^{[l]}}$$

For the final layer $l = L$ and $i = 1$:

$$\delta_1^{[L]} = \frac{\partial j(\Theta)}{\partial s_1^{[L]}}$$



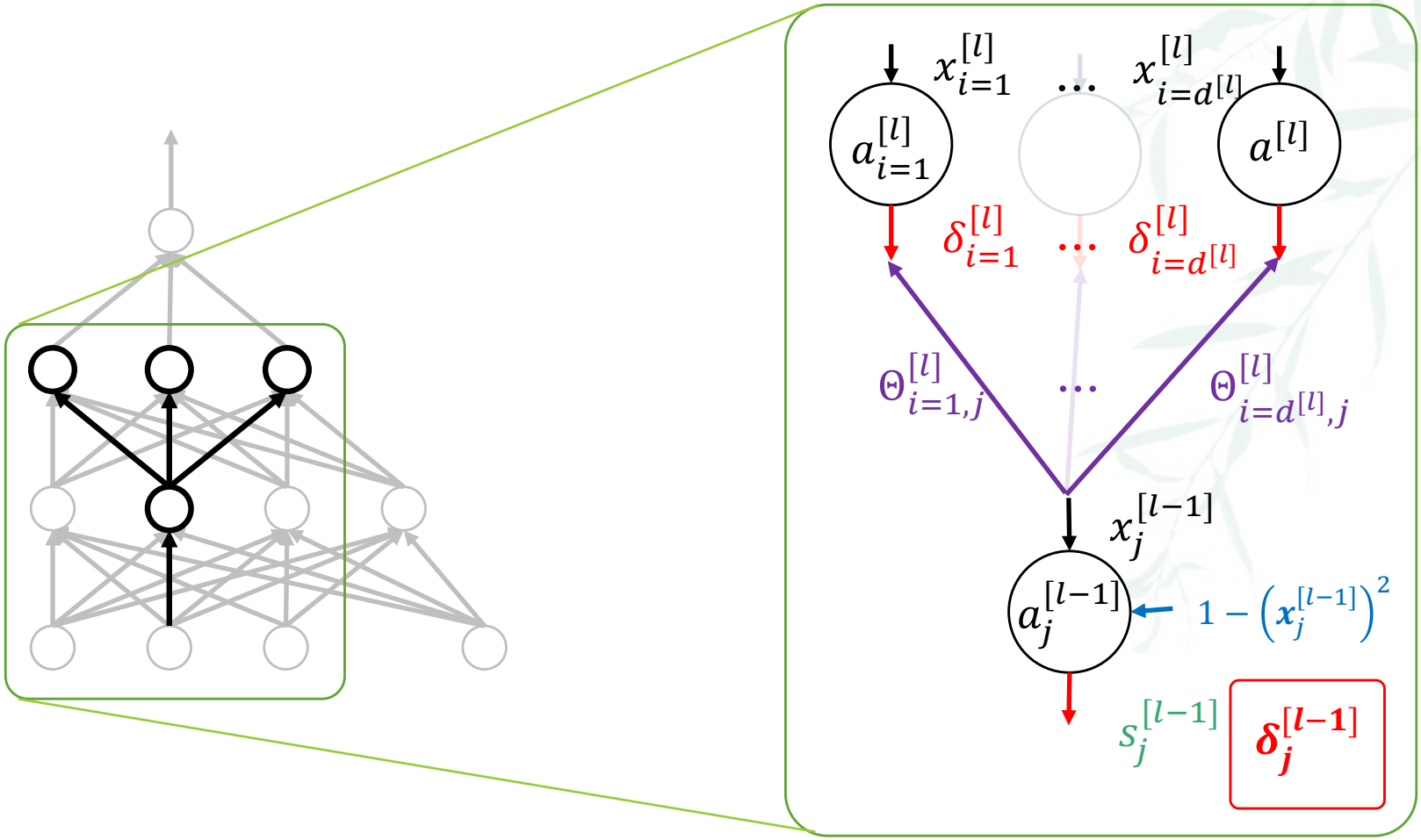
$$j(\Theta) = j(h(\boldsymbol{x}^{(*)}), y^{(*)}) = \left(a_1^{[L]} - y^{(*)}\right)^2$$

$$a_1^{[L]} = g\left(s_1^{[L]}\right)$$

$$g'(s) = 1 - g^2(s) \text{ for tanh}$$

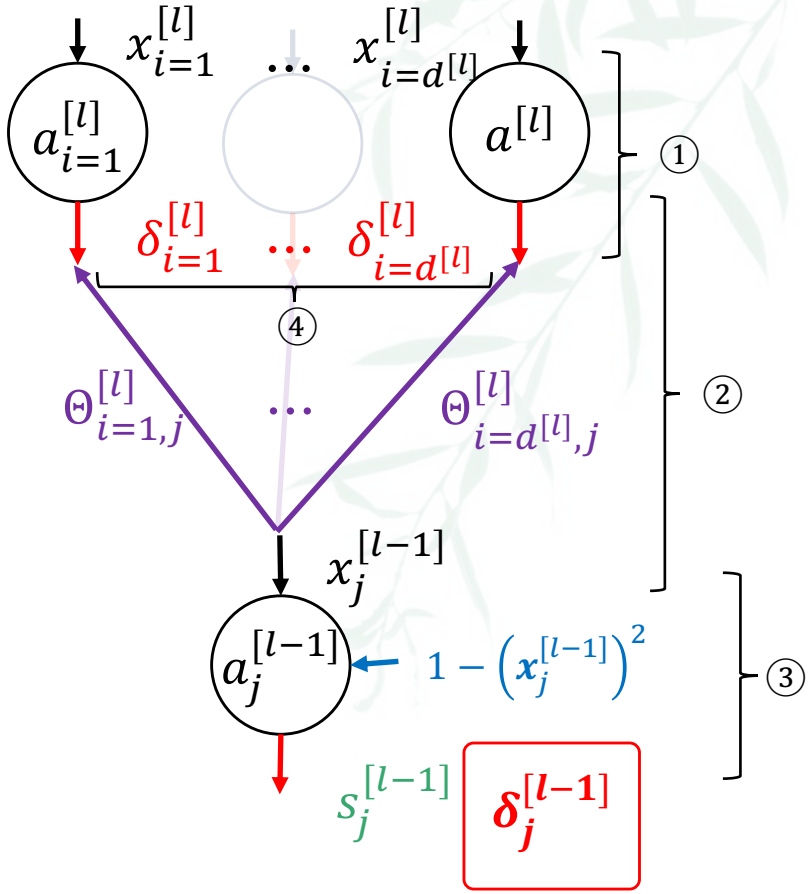
Back Propagation of δ

$$\boxed{\boldsymbol{\delta_j^{[l-1]}}} = \frac{\partial\, j(\Theta)}{\partial\, s_j^{[l-1]}}$$



Back Propagation of δ

$$\boxed{\boldsymbol{\delta_j^{[l-1]}}} = \frac{\partial\, j(\Theta)}{\partial\, s_j^{[l-1]}}$$



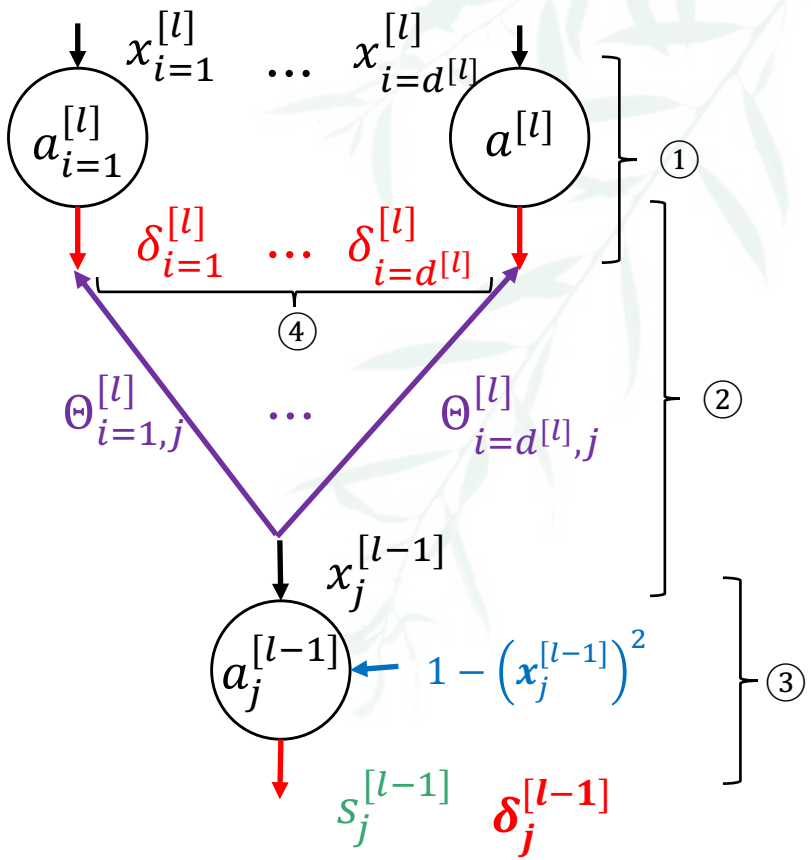
Back Propagation of δ

The view from a single unit $\boldsymbol{\delta_j^{[l-1]}} = \dfrac{\partial\, j(\Theta)}{\partial\, s_j^{[l-1]}}$

$$= \underbrace{\sum_i^{d^{[l]}}}_{④} \underbrace{\frac{\partial\, j(\Theta)}{\partial\, s_i^{[l]}}}_{①} \times \underbrace{\frac{\partial\, s_i^{[l]}}{\partial\, x_j^{[l-1]}}}_{②} \times \underbrace{\frac{\partial\, x_j^{[l-1]}}{\partial\, s_j^{[l-1]}}}_{③}$$

$$= \sum_i^{d^{[l]}} \delta_i^{[l]} \times \Theta_{i,j}^{[l]} \times \underbrace{g'(s_j^{[l-1]})}_{⑤}$$

$$\boldsymbol{\delta_j^{[l-1]}} = \underbrace{\left(1 - \left(x_j^{[l-1]}\right)^2\right)}_{⑤} \sum_{i=1}^{d^{[l]}} \Theta_{i,j}^{[l]} \delta_i^{[l]}$$



Backpropagation Algorithm

initialize all weights $\Theta_{i,j}^{[l]}$ ~~as $\vec{0}$~~ at random

for $t = 0, 1, 2, \ldots$ do

pick $(\boldsymbol{x}^{(*)}, y^{(*)}) \in \{1,2,\ldots,m\}$

Forward: Compute all $x^{[l]}$

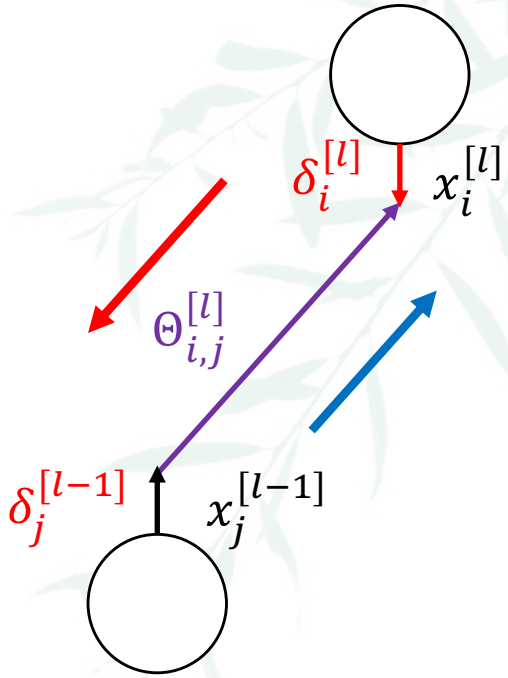
Backward: Compute all $\delta^{[l]}$

Update the weights:

$$\Theta_{i,j}^{[l]} \leftarrow \Theta_{i,j}^{[l]} - \eta\, x_j^{[l-1]} \delta_i^{[l]}$$

Iterate to the next step until termination

Return the final weights $\Theta_{i,j}^{[l]}$



Stuck? Try walking through the online tutorial by Matt Mazur.

Learning Nonlinear Transforms

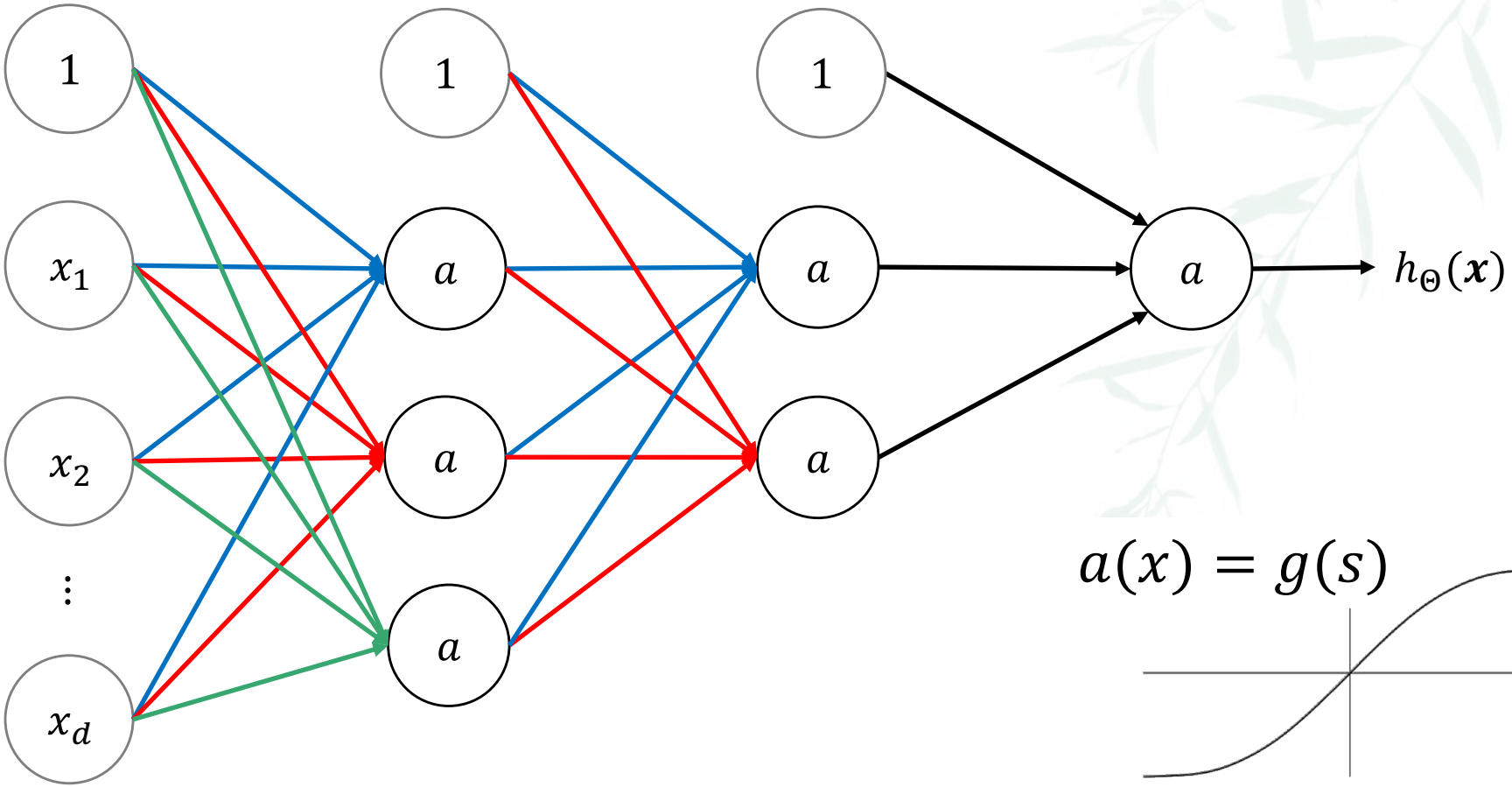
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

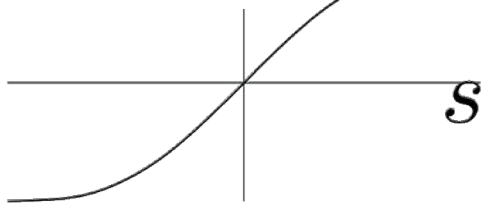
Hidden Layers

learned nonlinear transform

Interpretation?



$a(x) = g(s)$



FeedForward Example

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Forward Propagation

Without recurrence: Easy Peasy

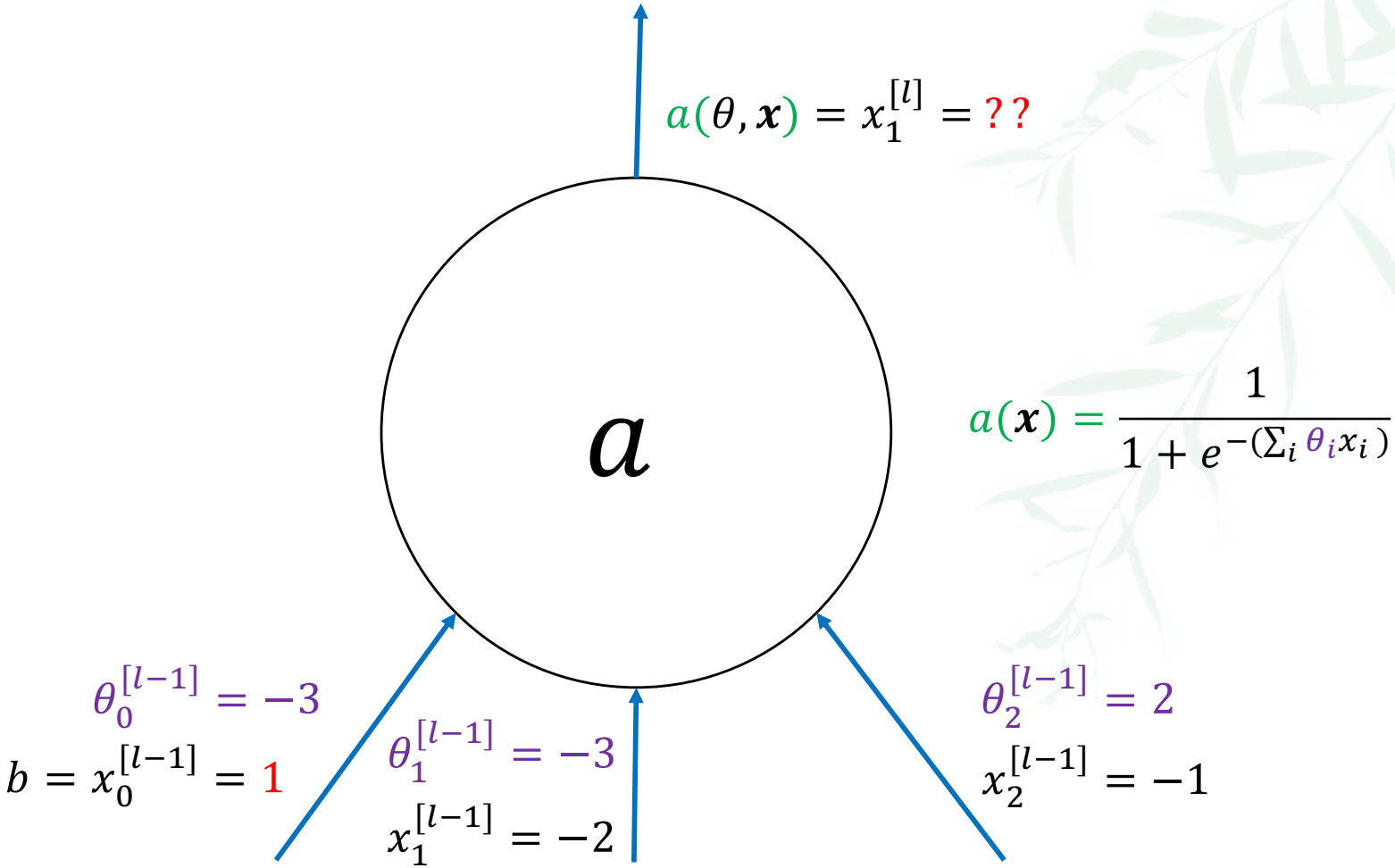


Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Feedforward Sigmoid Example

Envision calculations as part of a computation graph, with atomic operations.

$$a(\theta, \boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$

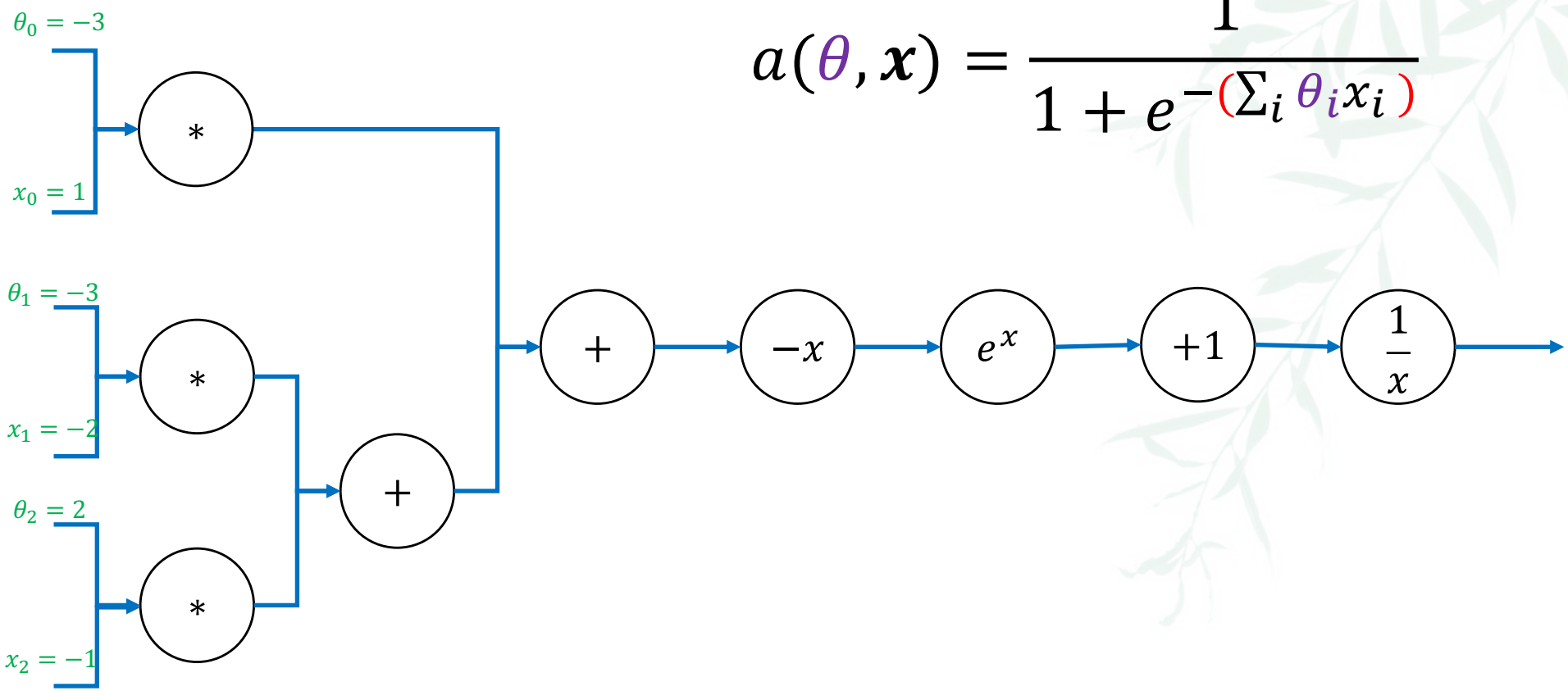
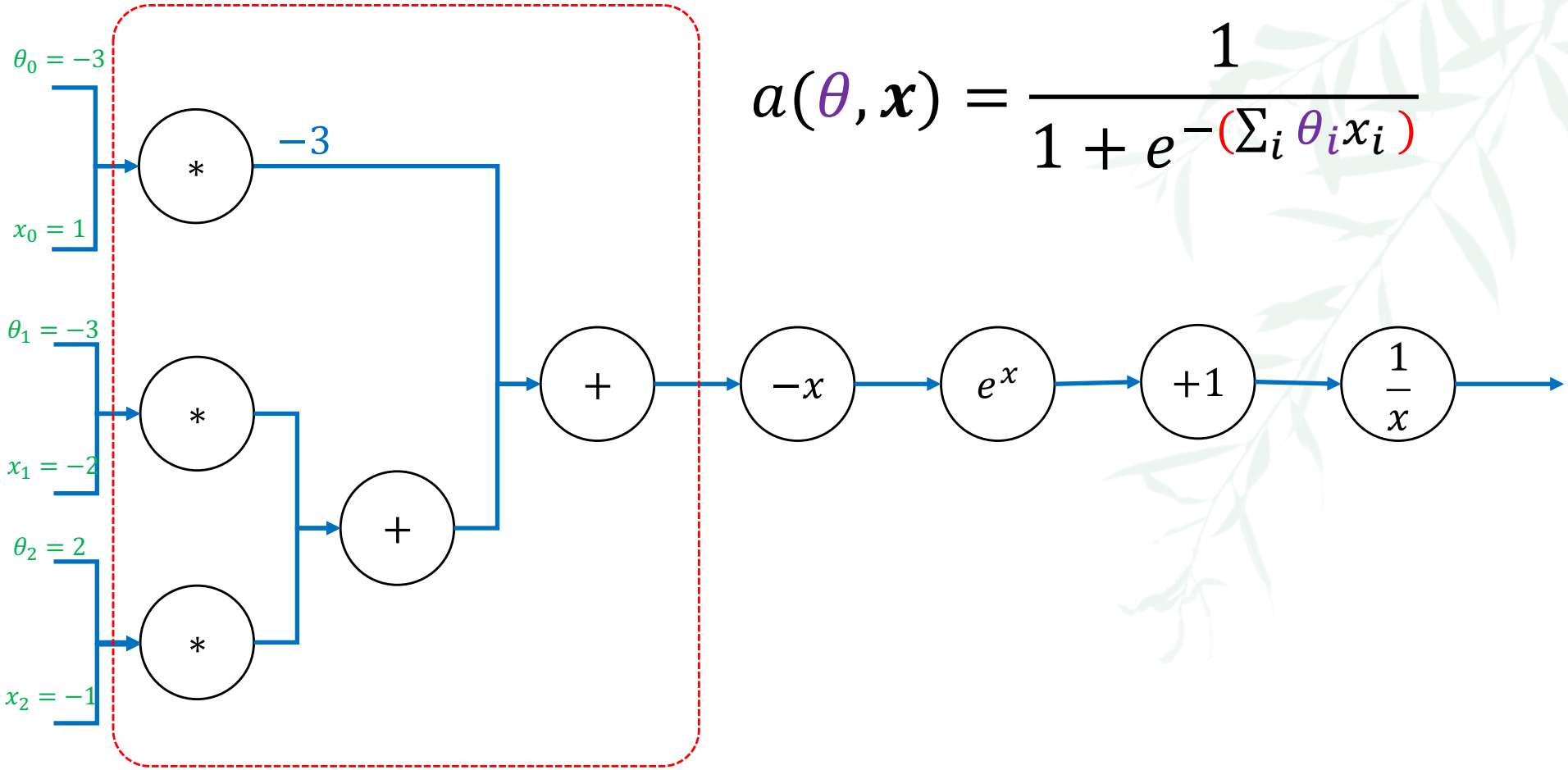


Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Feedforward Sigmoid Example



$$a(\theta, \boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$

Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Feedforward Sigmoid Example

$$a(\theta, \boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$

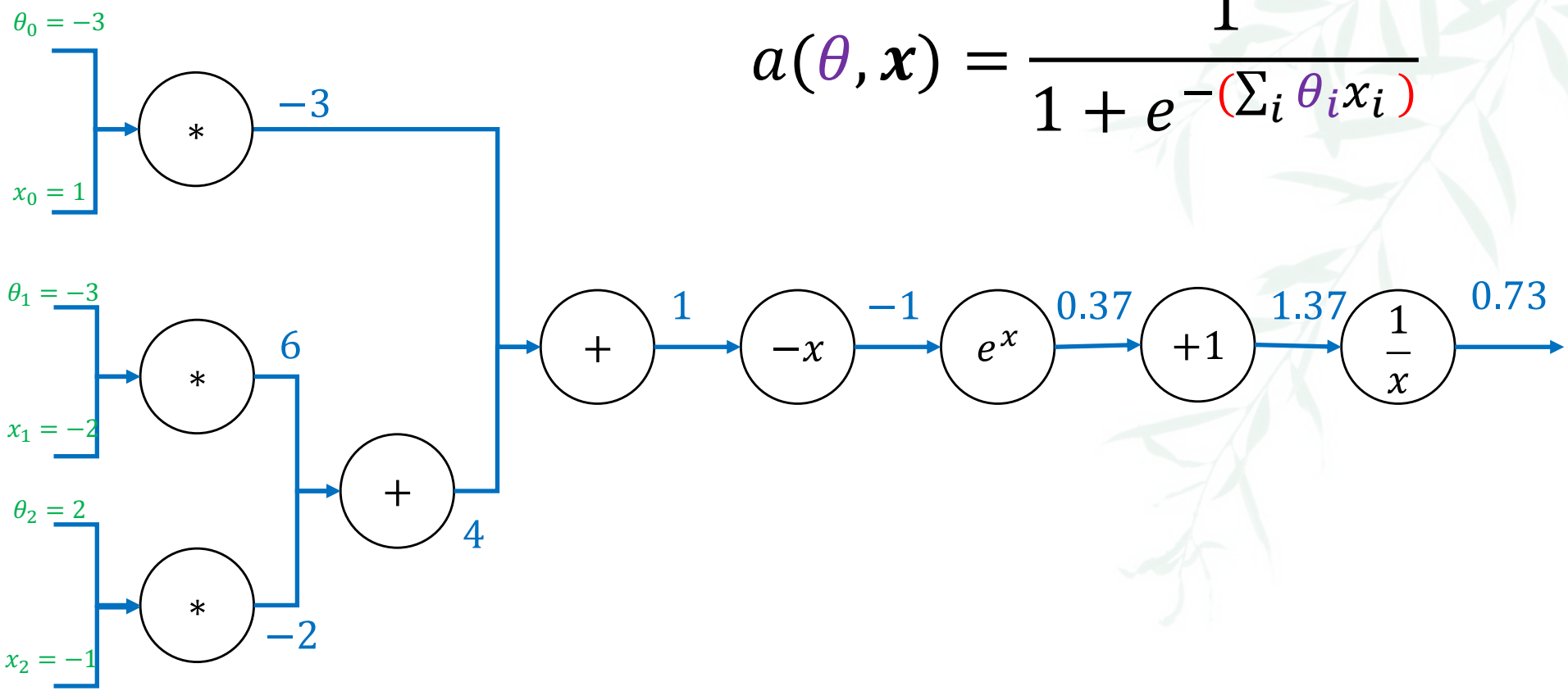


Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

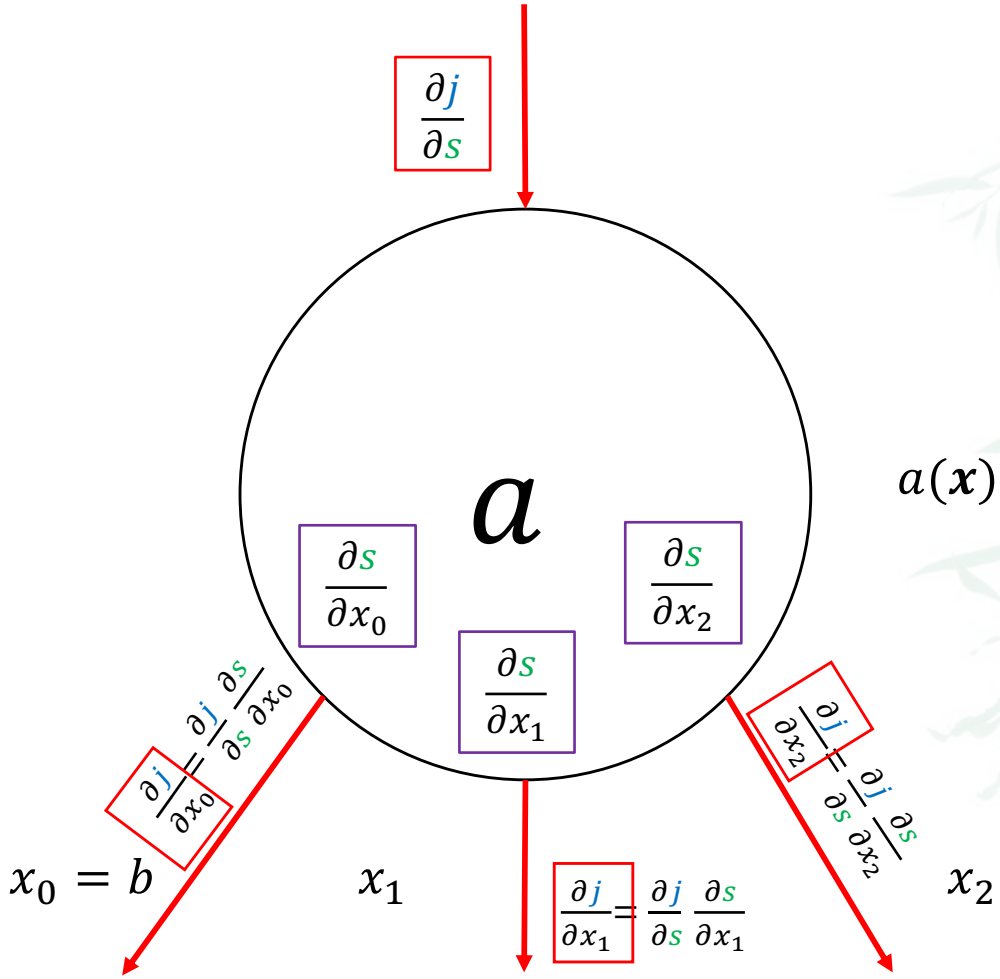
Backprop Example

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Chain Rule ≡ Backprop

Local Gradient: How the node views inputs with respect to outputs

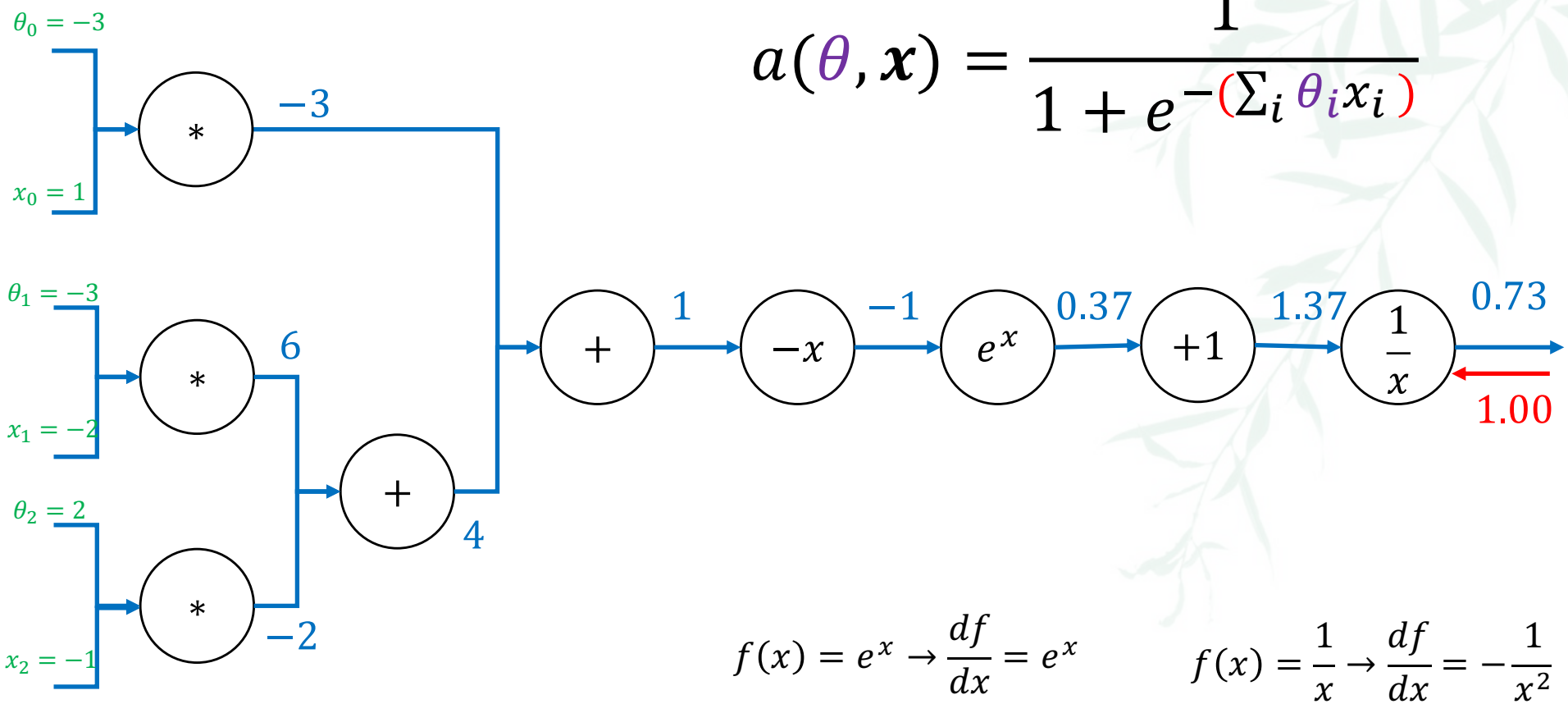


$$a(\boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$

Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Backprop Example

$$a(\theta, \boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$



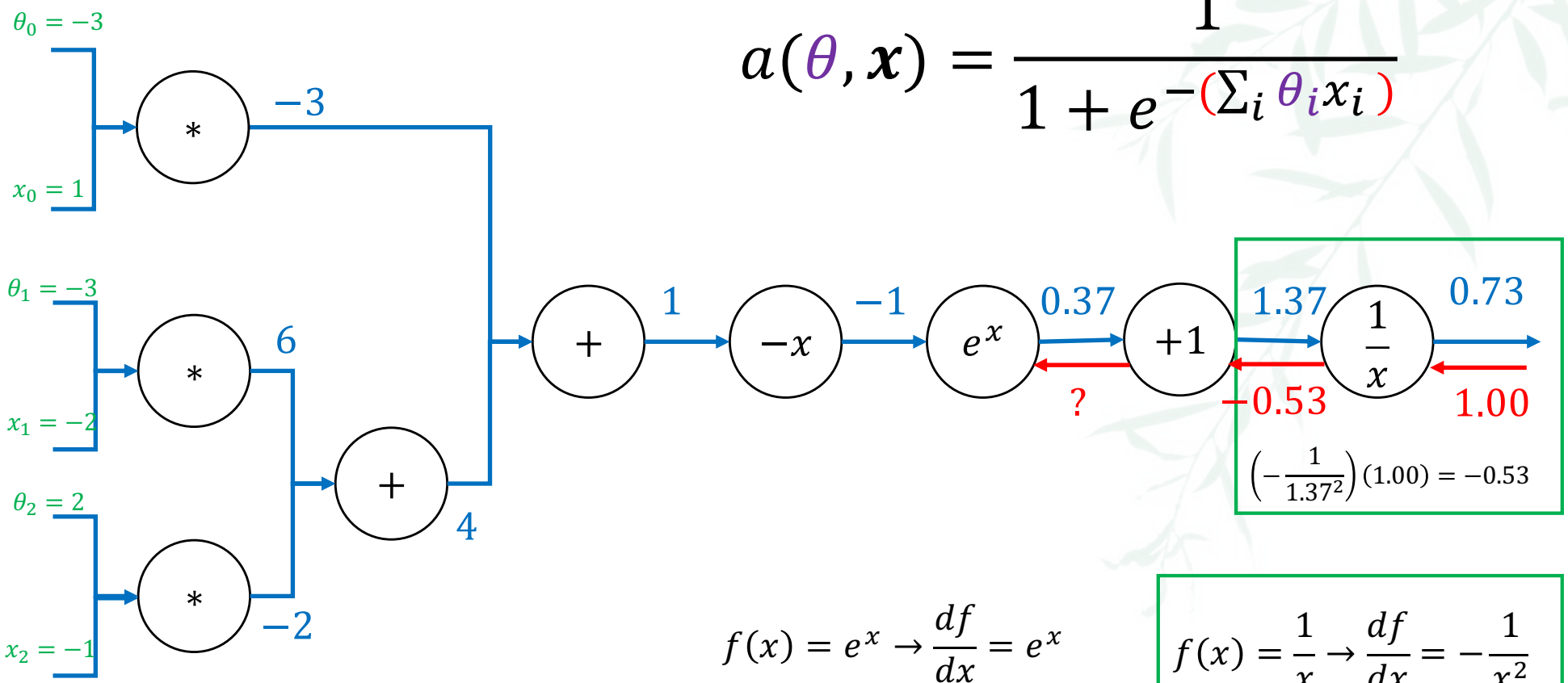
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x \qquad f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

$$f = ab \rightarrow \frac{\partial f}{\partial a} = b, \frac{\partial f}{\partial b} = a \qquad f_a(x) = ax \rightarrow \frac{df}{dx} = a \qquad f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Backprop Example

$$a(\theta, \boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

$$f = ab \rightarrow \frac{\partial f}{\partial a} = b, \frac{\partial f}{\partial b} = a$$

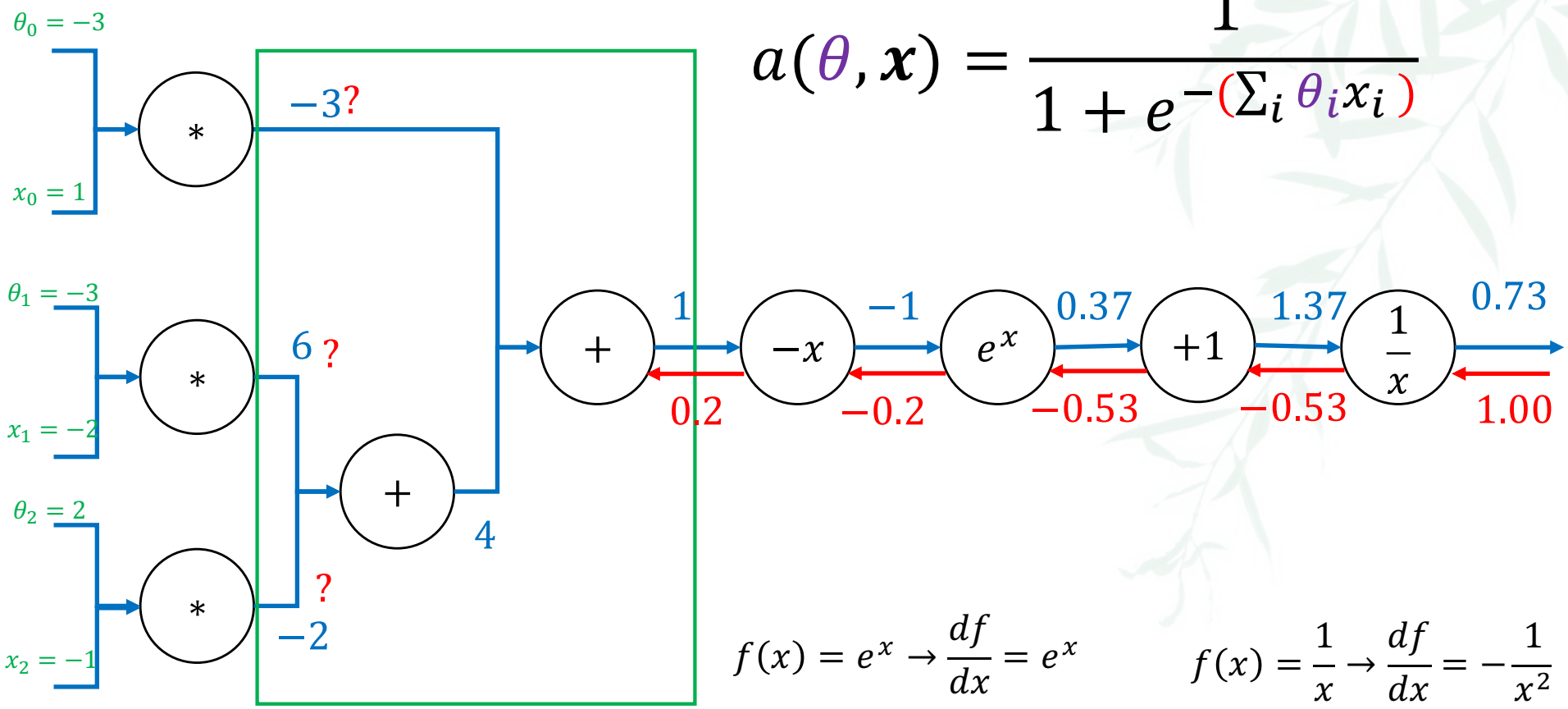
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Backprop Example

$$a(\theta, \boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

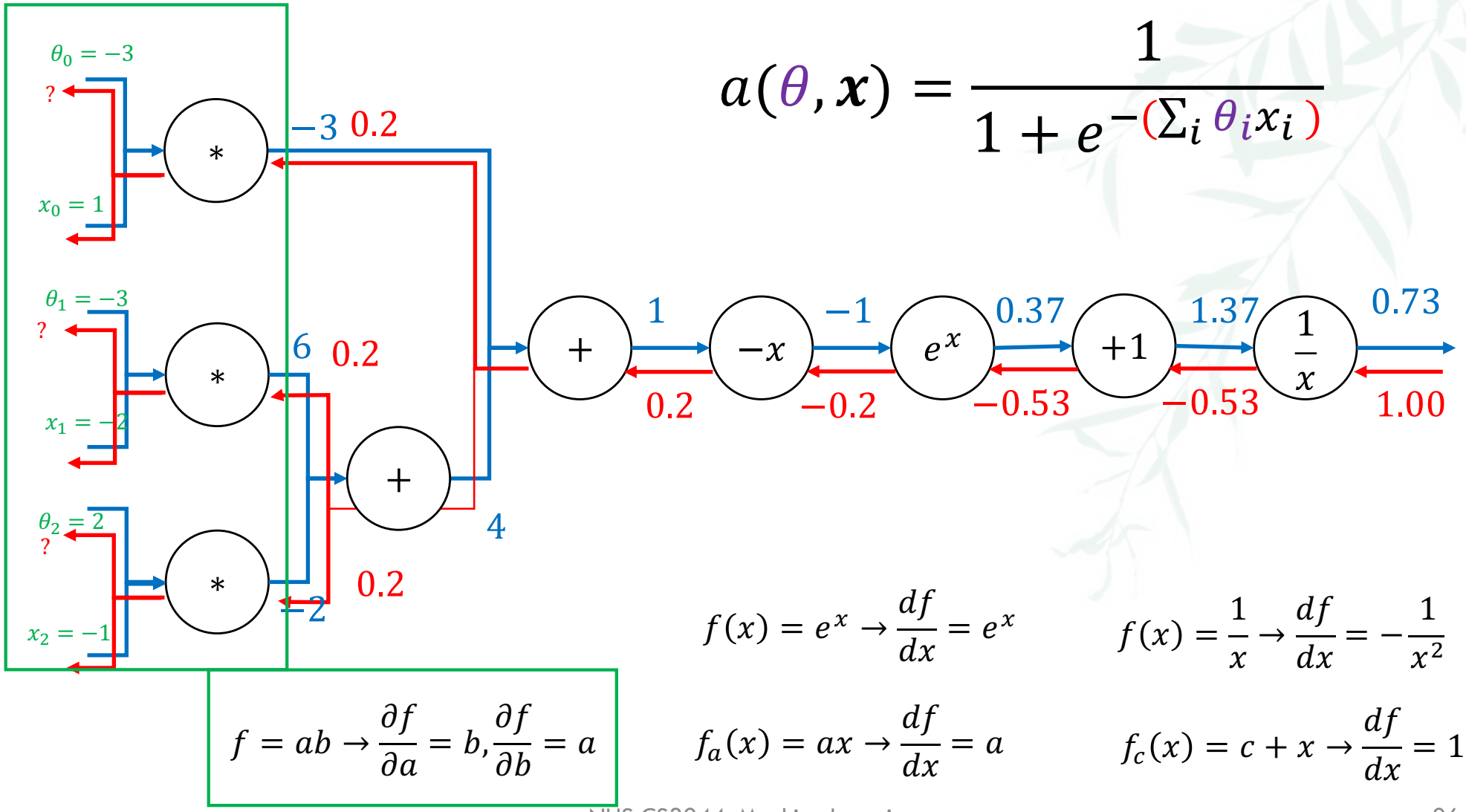
$$f = ab \rightarrow \frac{\partial f}{\partial a} = b, \frac{\partial f}{\partial b} = a$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Backprop Example



$$a(\theta, \boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$

$$f = ab \rightarrow \frac{\partial f}{\partial a} = b, \frac{\partial f}{\partial b} = a$$

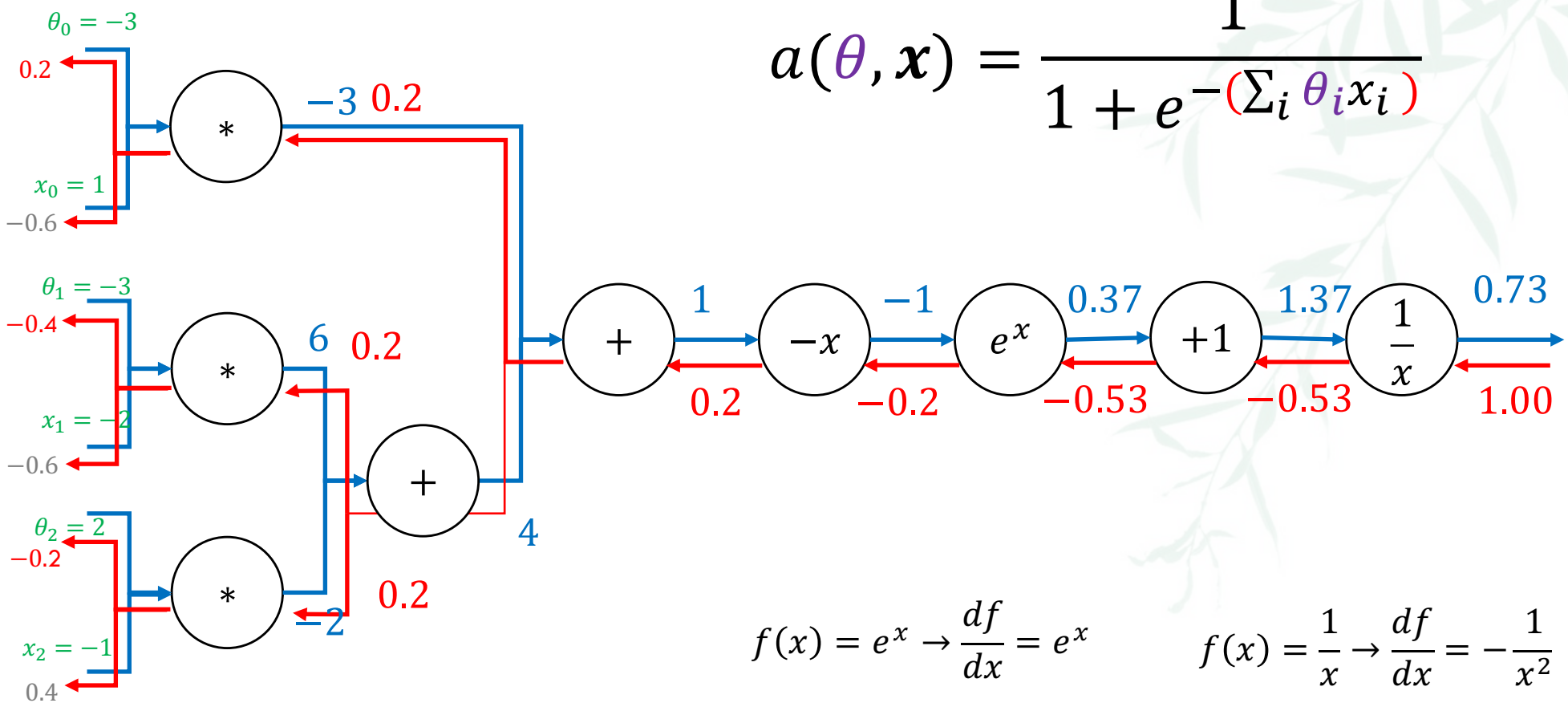
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x \qquad f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a \qquad f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Backprop Example

$$a(\theta, \boldsymbol{x}) = \frac{1}{1 + e^{-(\sum_i \theta_i x_i)}}$$

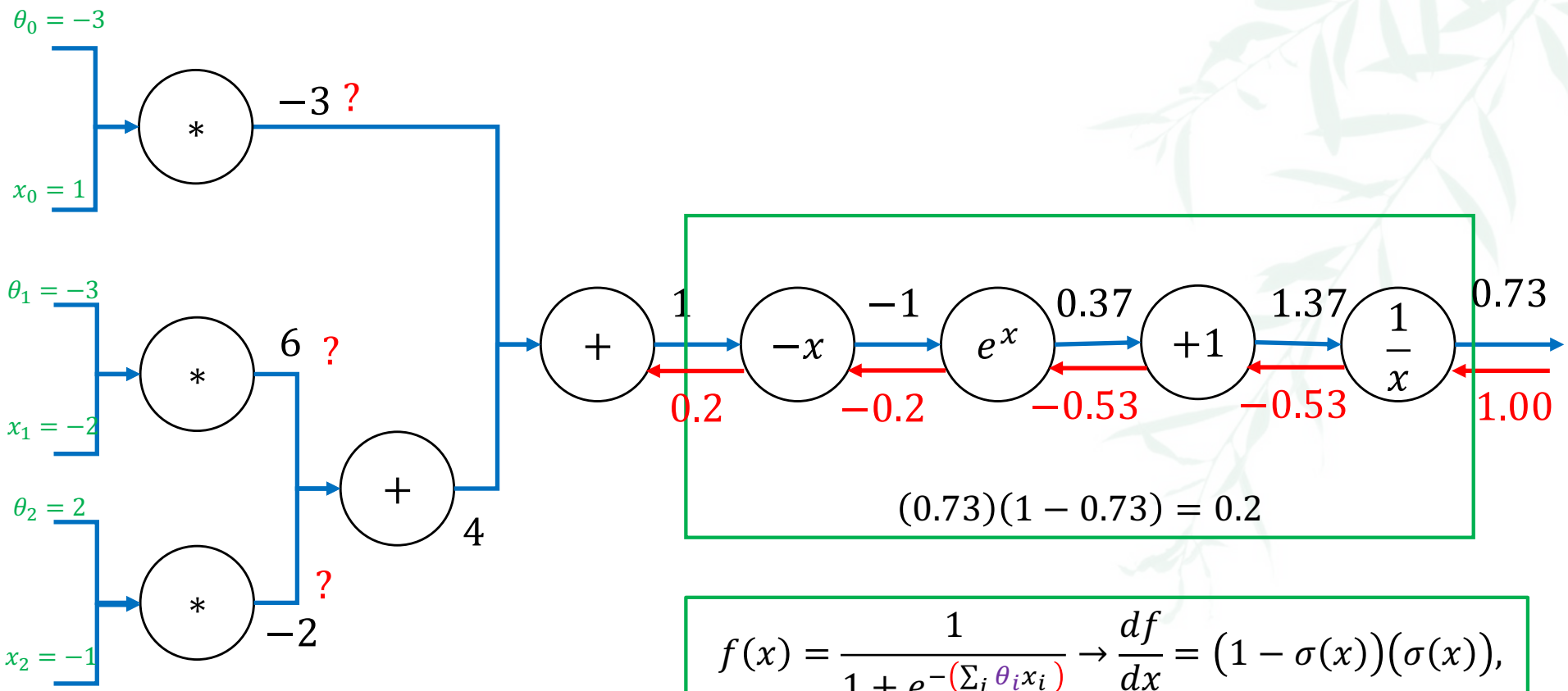


$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x \qquad f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

$$f = ab \rightarrow \frac{\partial f}{\partial a} = b, \frac{\partial f}{\partial b} = a \qquad f_a(x) = ax \rightarrow \frac{df}{dx} = a \qquad f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Sigmoid as a composition



$$f(x) = \frac{1}{1+e^{-(\Sigma_i \theta_i x_i)}} \rightarrow \frac{df}{dx} = \big(1-\sigma(x)\big)\big(\sigma(x)\big),$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$

Diagram credits for this section: Fei-Fei Li, Justin Johnson and Serena Yeung (Stanford CS231n)

Universal Approximation Theorem

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Summary: Universal Approximation

A single hidden layer NN of finite size can approximate any function

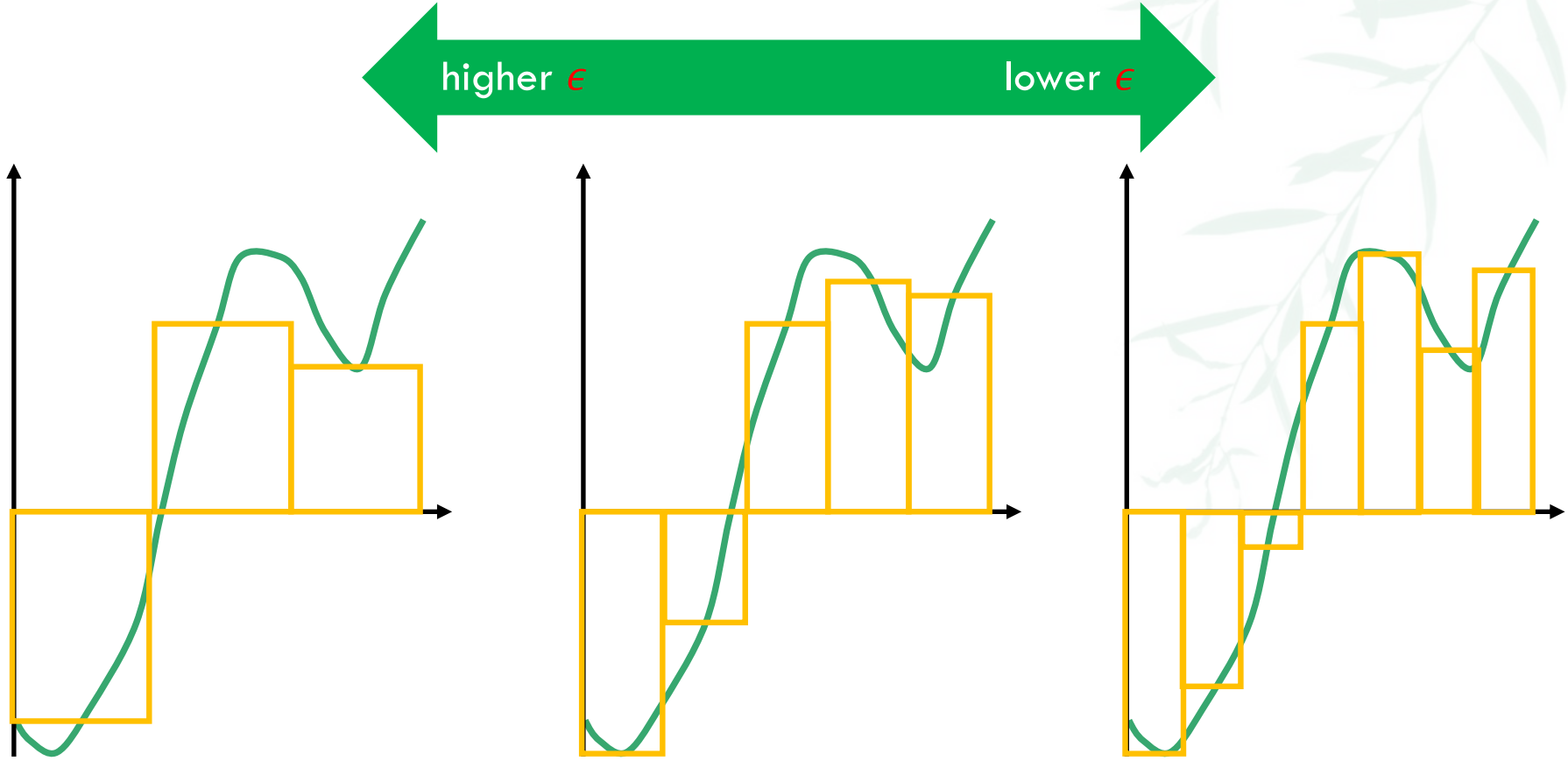
We can calculate $|h_\Theta(\boldsymbol{x}) - f(\boldsymbol{x})| < \epsilon$, for any ϵ that we wish. For lower ϵ, we'll need accordingly more units.

Even though we know that an NN exists that can compute $f(\boldsymbol{x})$, we may not have the means for its constructing or recognition.

Holds for the topology, non-linear activation function doesn't really matter.

A Graphical Proof

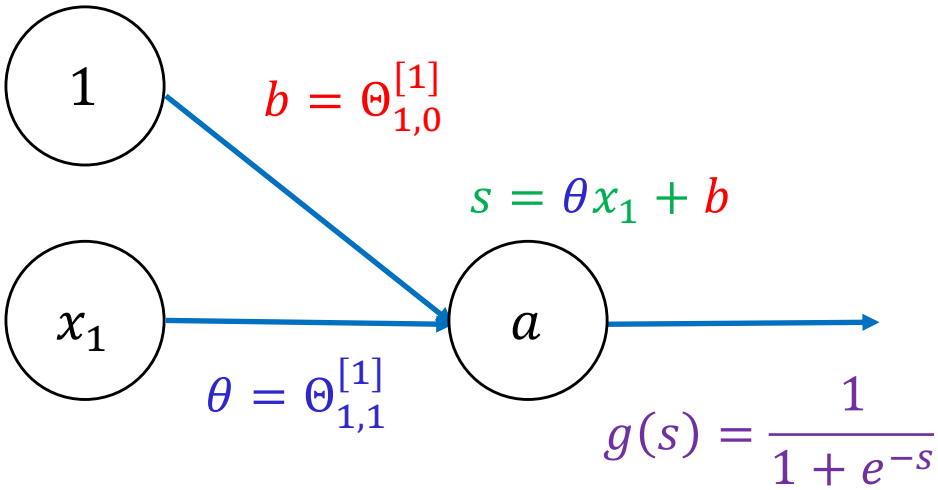
1. Build a bump function from linear units
2. Compose an arbitrary number of bumps to approximate $f(\boldsymbol{x})$.

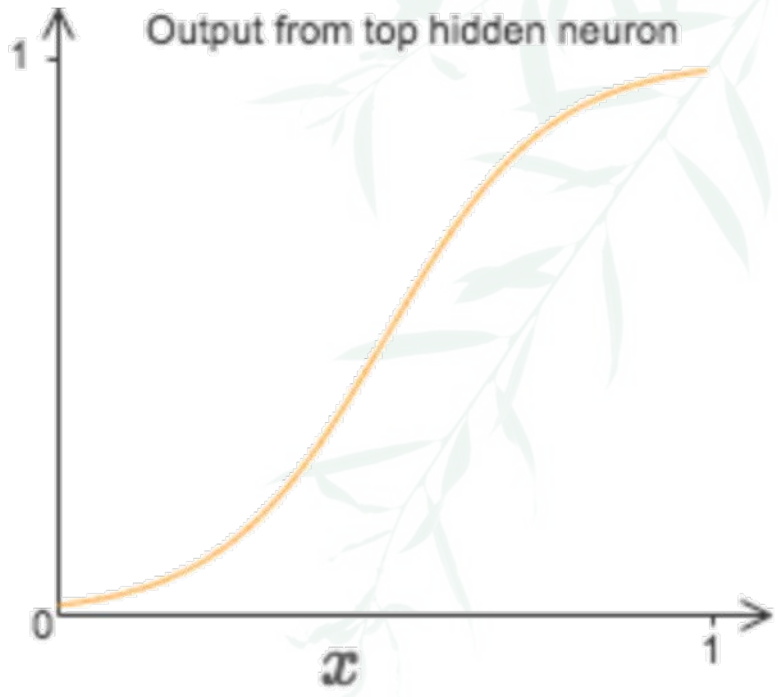


Adapted based on Michael A. Nielsen "NN and DL" 2015, Determination Press, CC By-NC 3.0

Recap: the single univariate unit

A single logistic unit computes a smooth $\mathcal{S}$-curve of the input.



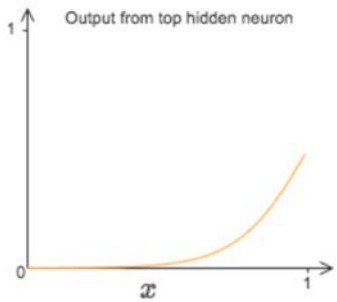


$g(x|\theta = 8, b = -4)$

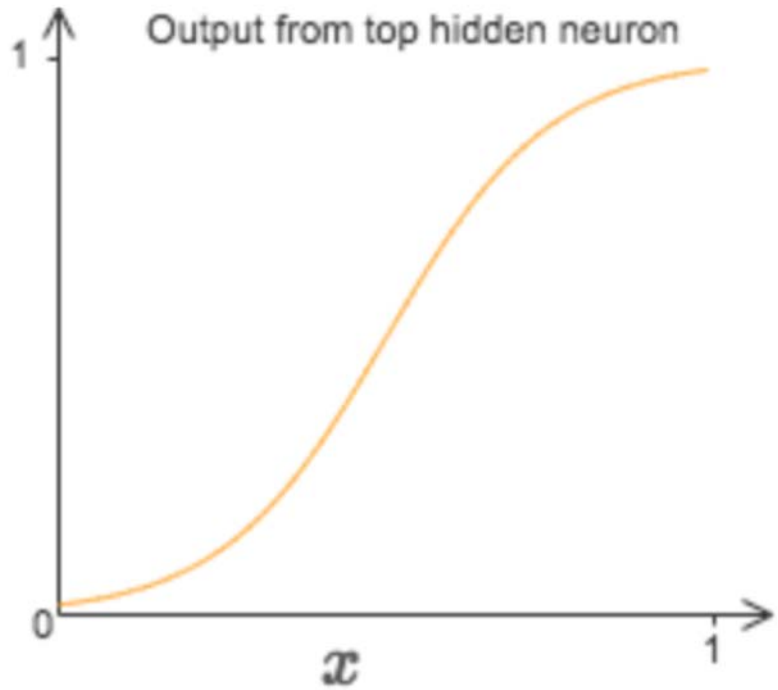
Adapted based on Michael A. Nielsen "NN and DL" 2015, Determination Press, CC By-NC 3.0

Varying θ and b

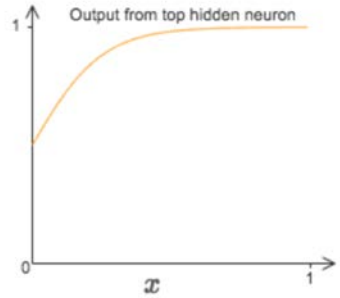
b varies the **shift** of the function



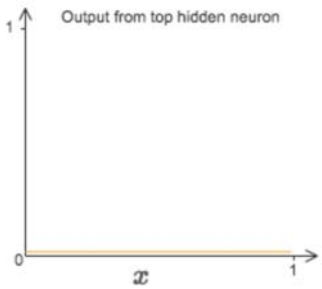
$g(x|\theta = 8, b = -8)$



$g(x|\theta = 8, b = -4)$



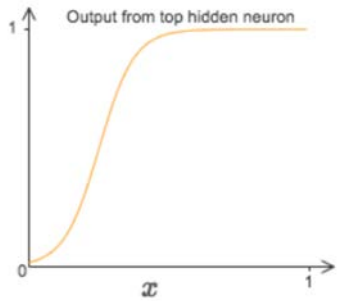
$g(x|\theta = 8, b = 0)$



$g(x|\theta = 0, b = -4)$

"linear"

θ varies the **slope** of the function



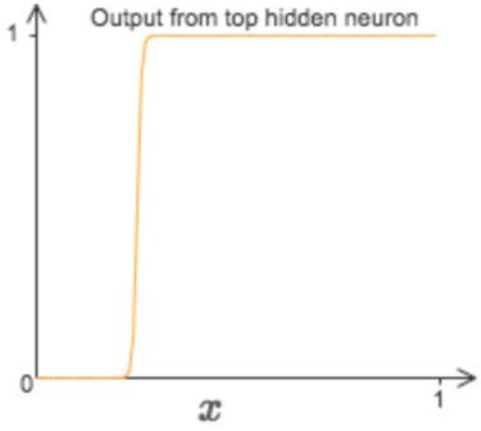
$g(x|\theta = 16, b = -4)$

"step"

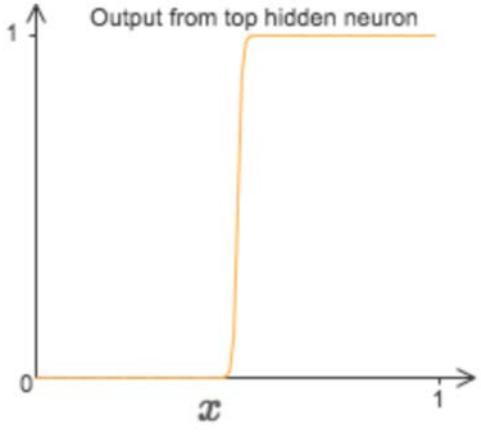
Adapted based on Michael A. Nielsen "NN and DL" 2015, Determination Press, CC By-NC 3.0

Let's work with step functions

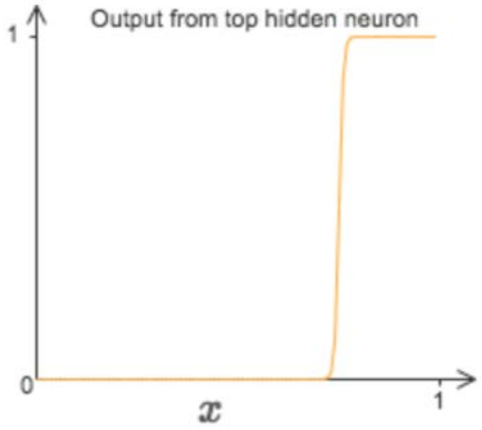
- Set θ high to get step-like output.
- Step occurs at $s = -\frac{b}{\theta}$.



$g(x|\theta = 200, b = -50)$



$g(x|\theta = 200, b = -100)$



$g(x|\theta = 200, b = -150)$

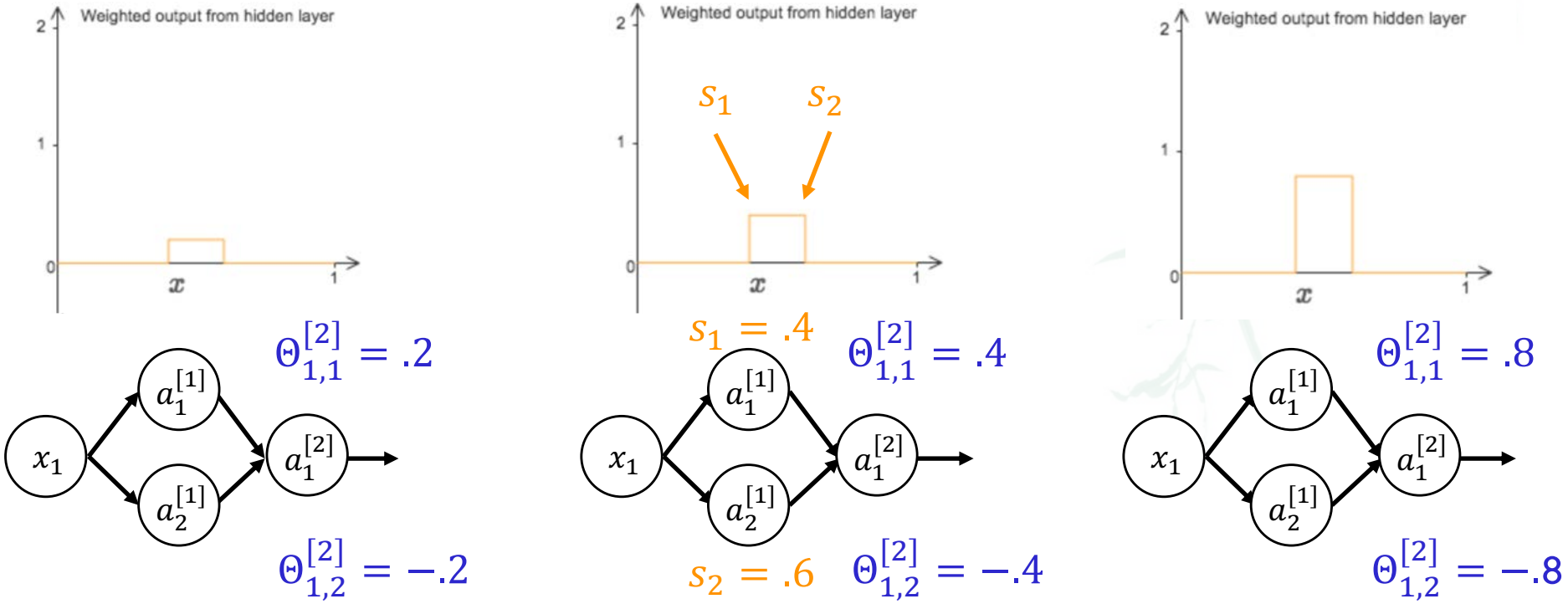
Adapted based on Michael A. Nielsen "NN and DL" 2015, Determination Press, CC By-NC 3.0

2 units: Computing a bump

One unit for step up and one unit for step down.

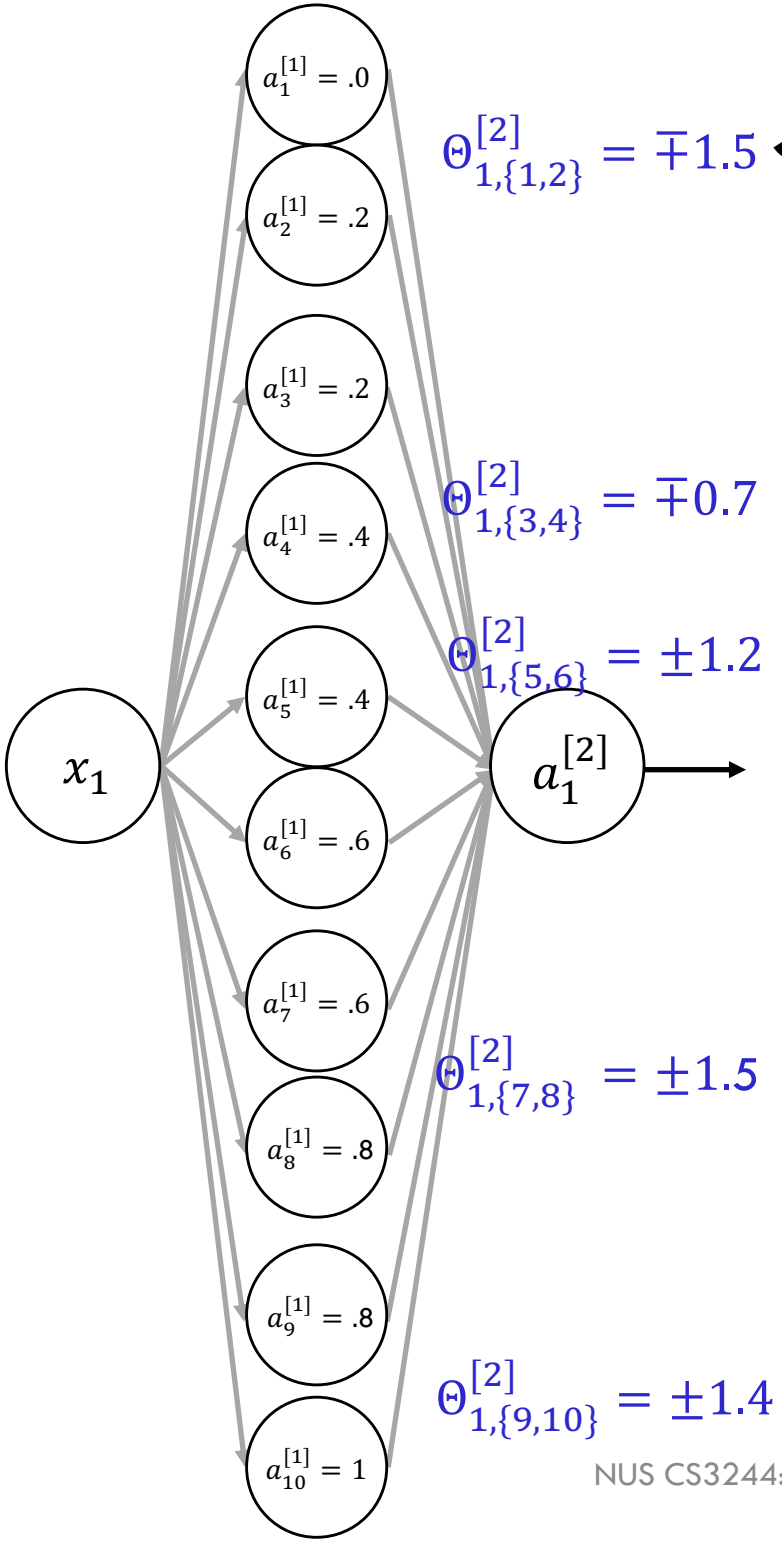
Weights at 2nd hidden layer additively compose both steps to form a bump.

Width determined by s_1 and s_2.



Adapted based on Michael A. Nielsen "NN and DL" 2015, Determination Press, CC By-NC 3.0

5 pairs of units



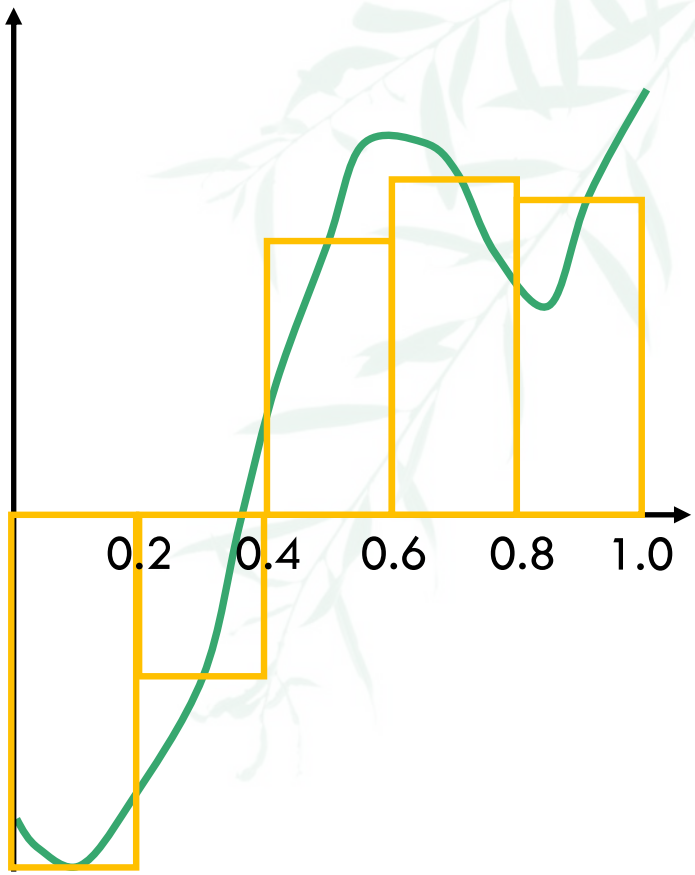
$\Theta^{[2]}_{1,\{1,2\}} = \mp 1.5$

$\Theta^{[2]}_{1,\{3,4\}} = \mp 0.7$

$\Theta^{[2]}_{1,\{5,6\}} = \pm 1.2$

$\Theta^{[2]}_{1,\{7,8\}} = \pm 1.5$

$\Theta^{[2]}_{1,\{9,10\}} = \pm 1.4$



Adapted based on Michael A. Nielsen "NN and DL" 2015, Determination Press, CC By-NC 3.0

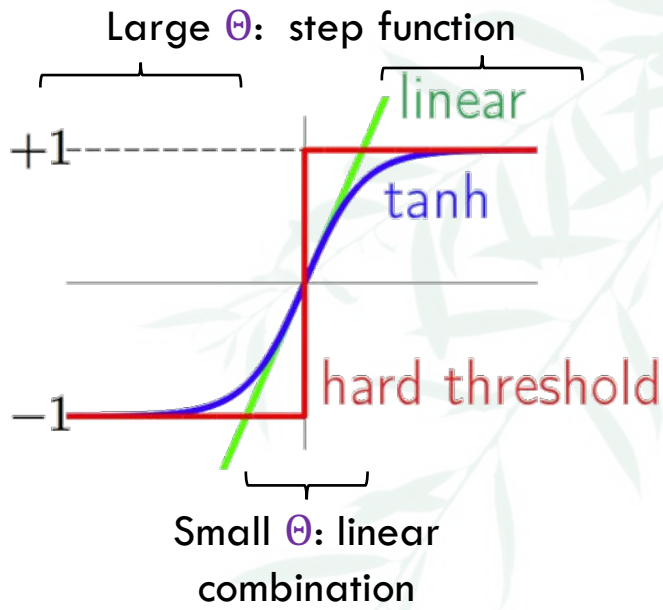
Regularizing NNs

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Regularization

1. Weight Decay: From linear to logical



2. Weight Elimination:

   Fewer weights ⟶ lower complexity, better generalization

Soft weight elimination:

$$\Omega(\Theta) = \sum_{i,j,l} \frac{\left(\Theta_{i,j}^{[l]}\right)^2}{1 + \left(\Theta_{i,j}^{[l]}\right)^2}$$

Small Θ pushed to zero; Big Θ left alone

Early stopping as a regularizer

Regularization through the optimizer

Exploring the weight space per iteration/epoch

Budget $C \uparrow$ per iteration

When to stop? Validation!

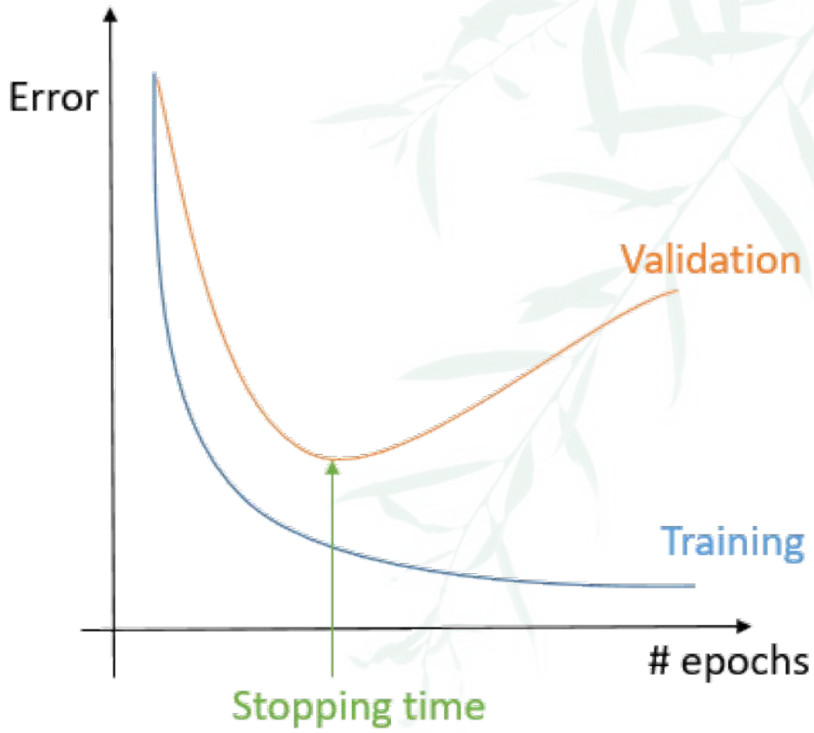


Diagram credits for this section: Martin Strobel CC BY 4.0